# Computer Graphics

## CSCI 490, CSCI 631

## Project 5

### Simple Shading

---

## Summary

The purpose of this assignment is to perform simple lighting calculations.

## Program Requirements

Your program should be able to

- set an ambient light.
- set multiple point lights and directional lights of arbitrary colors.
- set global parameters associated with rendering.
- apply surface shader functions to arbitrary objects. The shading functions that should be implemented are `matte`, `metal`, and `plastic`.

Additional scene files are available for this project.

## Things That Need to be Implemented and Modified — What to Do

This is a list of data structures and routines that need to be implemented or modified, and a suggested implementation order.

### Lights

You need a way to store the lights in the scene. There are three types of lights:

- Ambient light - This consists of an intensity in each of the primary color components, R, G, and B.
- Far Light - Each far light has a color intensity and a direction that the light is pointing.
- Point Light - Each point light has a color intensity and a position.

You may implement the lights any way you wish, but a class or struct is highly recommended.

Each scene may have several lights. These should all be globally available. There will only be one ambient light in a scene, but possibly several point lights and far lights. Globally available arrays of point lights and far lights is a simple way to store them. Your program should be able to handle at least 30 of each.

### New Global Variables

There is a lot of global information that needs to be available for lighting calculations.

- The lights and any associated information (number of each type of light).
- An ambient coefficient, a diffuse coefficient, and a specular coefficient. Default values should be 1, 0, and 0 respectively.
- A surface color. This will take over the role of the current drawing color. It should have a default value of white.
- A specular color. This is the color that will be used in specular lighting calculations. It has a default value of white.
- A specular exponent. The default value is 10.0.
- Four flags. Three of these indicate the existence of information associated with each attributed point vertex of a polygon: a `vertex_color_flag`, a `vertex_normal_flag`, and a `vertex_texture_flag`. The fourth flag is an interpolation flag. It determines whether or not interpolated values are used in lighting calculations. By default, it is true.
- A viewing vector. This is the direction from a surface point, to the eye and is calculated for each surface point processed, as needed.
- A polygon normal. This is the surface normal for a polygon. It may or may not be used depending on whether a constant polygon normal value is used for lighting or an interpolated normal value is used.
- An attributed point, `surface_point_values` which is set with interpolated values during polygon scan conversion. These values are used during lighting calculations.
- A pointer to a function that takes a reference to a color and returns `void`. This is the `surface_shader`. You should have three functions, `matte`, `metal`, and `plastic` each of which takes a reference to a color and returns `void`. The `surface_shader` should have a default value of the `matte` function.

## Normal Transformation

All polygon normals must be transformed by a *normal transformation* matrix. This is similar to the current transformation matrix. It is most easily computed in the `rd_translate`, `rd_rotate`, and `rd_scale` routines. After combining the proper transformation matrix on the right of the current transformation matrix, combine the inverse of the initial transform on the left of the normal transformation matrix. Points are applied to the current transformation matrix on the right. Normals are applied (multiplied) to the normal transformation on the left. This matrix must be saved and restored in all `XformPush` and `XformPop`.

## Polygon Pipeline Changes

Some additions need to be made to the polygon pipeline.

- After the object coordinate part of the attributed point has been converted to world coordinates, the values should be saved in the `WORLD_X`, `WORLD_Y`, and `WORLD_Z` fields of the attriubuted point.
- The surface normal needs to be converted world coordinates as well. The object coordinate form of the normal is found in the attributed point and extracted. It is multiplied by the normal transformation matrix and stored back in the appropriate fields of the attributed point.
- Be sure to set the constant field in the attributed point to 1.0
- In the loop processing the clipped vertices before scan conversion, after the x and y coordinates have been normalized (divided) by w, the other attributes in the point, from the constant on, need to be normalized as well.
- Before the scan conversion routine is called, the global polygon normal, which is in object coordinates, needs to be converted to world coordinates.

## Polygon Scan Conversion Changes

There are some changes that needs to be made to the polygon scan conversion code in order for shading to take place. Fortunately, all of the changes need to be made in the routine that fills between the edges.

In this routine, in the loop that actually sets the pixels, the following changes need to be made to determine the color of the pixel to be plotted:

- The attributes in the `value` variable need to be stored in the global `surface_point_values`variable and divided by the interpolated constant in that value. This will give the proper interpolated world coordinate values for the attributes. The attributes in `value` should not be changed.
- The `surface_shader` function should be called to compute the color to be applied to the screen
- The screen x, y, and z, along with the computed color are used to plot the pixel on the screen
- After plotting, the values along the scanline should be incremented, as before.

## Objects

- Change your `cube`, `cone`, `cylinder`, `disk`, and `sphere` routines.

  At the beginning of each object, you should turn off the `vertex_color_flag` and the `vertex_texture_flag`. You should turn on the `vertex normal flag`.

  Your routines should calculate a surface normal for each vertex and place it in the appropriate fields of the attributed point before it is passed to the polygon pipeline. The surface normal values should be obtained from what you know about the properties of the object being rendered. For example, the normal vector to a circle or a sphere is the vector obtained from going from the origin through the surface point. In other words, for a circle or sphere, the normal vector has the same coordinate values as the vertex position.

  Immediately before sending the last point of a polygon to the polygon pipeline, a normal vector for that polygon should be calculated and stored in the global `poly_normal` variable. This should be done regardless of whether or not vertex normals have been calculated or provided.

- Your polyset routine should be changed to simply copy the values from the vertex list into an attributed point that will be passed through the pipeline. The polygon normal (which should be calculated and stored in the global surface_normal variable) can be found by taking the cross product of two of the polygon edges. Order is important here. If you see the vertices traversed in counter-clockwise order, the normal should be coming toward you.

## Other functions

You need to implement the `rd_option_bool` method if you haven't already. This method takes a name of an option and its value. The implementation is simply a large if / elseif construct that recognizes the name and does the appropriate thing. For this assignment, the option that you need to recognize is the `Interpolate` option. It should set a global interpolation flag to the appropriate value. By default the global interpolation flag should be true.

## Utility routines

A routine `float clamp(float a, float x, float b)` which returns the value of x clamped to fall in the range of a to b is extremely helpful here.

A routine to take a vector I and reflect it about a vector N, resulting in a vector R is also very useful.

## Some shading tips

- Each shader needs to determine where the surface color and surface normal information are coming from. If the global interpolation flag is on and the appropriate vertex flag is on, then the information can be

found in the `surface_point_values` variable. Otherwise, it is found in the separate global variables which are constant for the polygon.
- In diffuse lighting effects, if the dot product of the surface normal N and the light vector L is less than 0 then no light should be added (or subtracted) from the accumulated light.
- When calculating diffuse effects, it's tempting to see if the dot product of the viewing vector V and the surface normal N is less than zero, i.e., if the surface is actually visible. In actuality, the effort needed to calculate the viewing vector V and do the test often matches the lighting calculation, so it's not worth it.
- When calculating specular effects, if the dot product of V and R is negative, then no light should be added (or subtracted) from the accumulated light.
- When calculating specular effects, check to see if the dot product of the viewing vector V and the surface normal N is less than 0. If so, then the view is of the back of the polygon and should not have any light reflecting off it. No specular lighting effects should be calculated in this case.
- The really ambitious can note that the dot product of V and the reflected light vector is the same as the dot product of the light vector and the reflected view vector. This can result in a nice computational savings because the reflected view vector need be calculated only once per pixel, but the reflected light vector must be calculated for every light for every pixel. When using this approach, the tests above must be strictly observed. There are some tricky cases here.

### Other Notes

- The lights are independent for each frame. The stored lights should be reset at the beginning of each frame/world block.

### Suggested Implementation Order

- Implement the parser functions that just set global variables. Make sure that the following global variables are initialized with proper default values when the program starts: All default colors should be white. The default ambient light is 1.0 in all channels. There are no point or directional lights. The default specular exponent is 10. The default surface shader should be the matte function.
- Implement the light functions. Intensity parameters for each light should simply be multiplied by the color before being stored in the light color.
- Implement the matte function to return the global surface color.
- Change `rd_color()` to set the `surface_color` variable as well as the drawing color.
- Modify your sphere, cone, cube, cylinder, disk and polyset routines.
- Change the matte function to use ambient lighting only. Try a simple scene file with a sphere. Change the light color. Change the ambient coefficient.
- Change the matte function to handle diffuse lighting calculations as well.
- Implement specular lighting calculations.
- Implement the metal and plastic shaders.

## Pass off

At pass off time, show your system running your version of the test scene files. Be prepared to modify the scene files and regenerate images during the pass off.

As always, make a scene of your own, and show the resulting image at passoff.